

Machine Learning

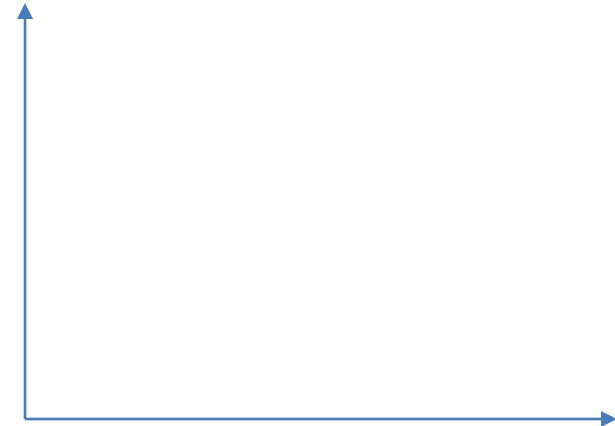
Instructor: Hafiz Abdul Rehman

Lecture 27: Unsupervised Learning

K-means clustering

K-means searches for a pre-determined number of clusters within an unlabeled multidimensional dataset. It accomplishes this using a simple conception of what the optimal clustering looks like:

- The "cluster center" is the arithmetic mean of all the points belonging to the cluster – the prototypical instance of that subpopulation
- Each point is closer to its own cluster center than to other cluster centers
- Hard, flat, polythetic clustering
 - Data partitioned into K sub-populations
 - K must be specified, K -means cannot find it
- Uses
 - Discover classes in an unsupervised manner
 - Cluster images of handwritten digits ($K=10$)
 - T-shirt sizes (S/M/L)
 - Regularization
 - Dimensionality reduction (clusters = "latent factors")
 - Represent each instance with its cluster number: Vector Quantization
 - Related to kernels in linear classifiers



- <https://jakevdp.github.io/PythonDataScienceHandbook/05.11-k-means.html>
- Victor Lavrenko, https://www.youtube.com/watch?v=c_oCcpN0_0s&list=PLBv09BD7ez_6lYVoZ1RzVcOPIT5Lfjo0Y&index=4



K-means: Lloyd/Forgy

method

(from M. Mitchell, CMU, <http://www.cs.cmu.edu/~gustrin/Class/10701-S05/slides/EM-MixGauss4-4-2005.pdf>)

Algorithm

Given data $\langle x_1, x_2, \dots, x_n \rangle$, and K

Initialize $\mu_1, \dots, \mu_K$ randomly (the K centroids)

Repeat until convergence:

1. Assign each point x_i to the cluster C_j with the closest mean μ_j

$$\operatorname{argmin}_j D(x_i, \mu_j)$$

2. Calculate the new mean for each cluster:

$$\mu_j \leftarrow \frac{1}{|C_j|} \sum_{x_i \in C_j} x_i$$

- The algorithm has converged when the assignments no longer change (or after predefined number of iterations).
- The algorithm is not guaranteed to find the optimum



EM Intuition

- Choose starting guesses of averages of each cluster
- Repeat until converged:
 - ***E-step*** (label out individuals)
 - For each point (individual), find the probability of membership in each cluster
 - ***M-step*** (update definition of the collectives)
 - For each cluster, update its definition (average, variance etc.) based on all newly included data points



EM Intuition

- **Goal:** Suppose we want to cluster our class into 2 clusters by height
- **Expectation-Maximization**
 - Start with two random clusters, say with means 4ft and 7ft
 - **E-Step:** Assign all students to one of the clusters based on distance
 - e.g. a student with height 5ft7in is 1ft5in away from 7ft and 1ft7in away from 4ft cluster: So assign to 7ft cluster
 - Assign every student of the class to one of the clusters
 - **M-Step:** Recalculate new means based on all cluster members labelled in the E-Step
 - The 7ft may come down e.g. to 6ft3in and the 4ft may go up to e.g. 5ft2in
 - Repeat until convergence

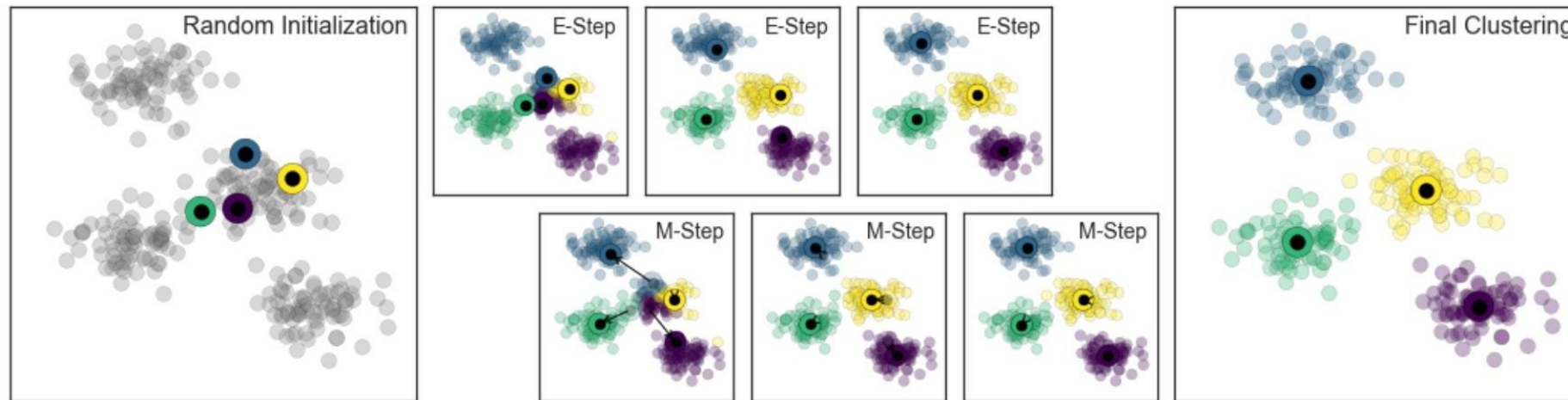


EM: K-means

clustering

• <https://jakevdp.github.io/PythonDataScienceHandbook/09.11-k-means.html>

- Guess some cluster centers
- Repeat until converged
 - **E-Step**: assign points to the nearest cluster center
 - **M-Step**: set the cluster centers to the mean



EM: GMM clustering

<https://jakevdp.github.io/PythonDataScienceHandbook/05.12-gaussian-mixtures.html>

- Choose starting guesses for the location and shape
- Repeat until converged:
 - ***E-step***: for each point, find weights encoding the probability of membership in each cluster
 - ***M-step***: for each cluster, update its location, normalization, and shape based on *all* data points, making use of the weights



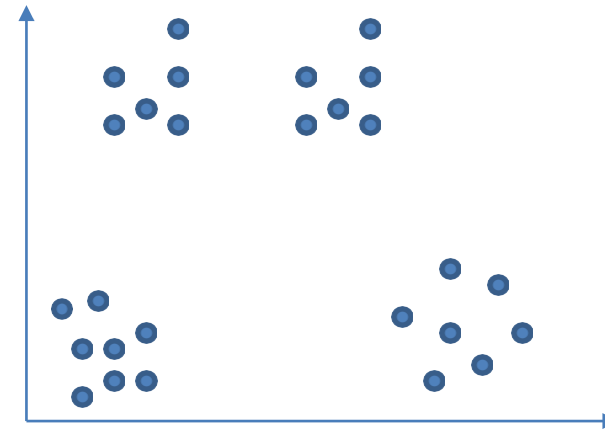
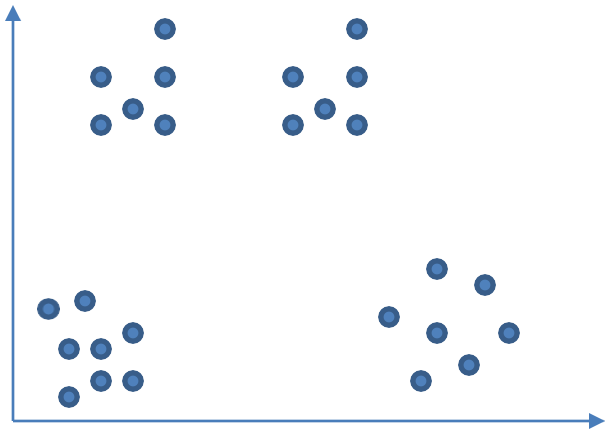
K-means in action

(<http://shahel.in/trigolo.html>)
K-means clustering.

The data points. Click the picture to continue.



Is a particular clustering good or bad?



The K-means Objective

(Tom M. Mitchell, CMU, <http://www.cs.cmu.edu/~guestin/Class/10701-S05/slides/EM-MixGauss4-4-2005.pdf>)

- Given data $\langle x_1, x_2, \dots, x_n \rangle$, and K , assign each x_i to one of K clusters, $C_1 \dots C_K$, minimizing:

$$J = \sum_{j=1}^K \sum_{x_i \in C_j} D(x_i, \mu_j)^2$$

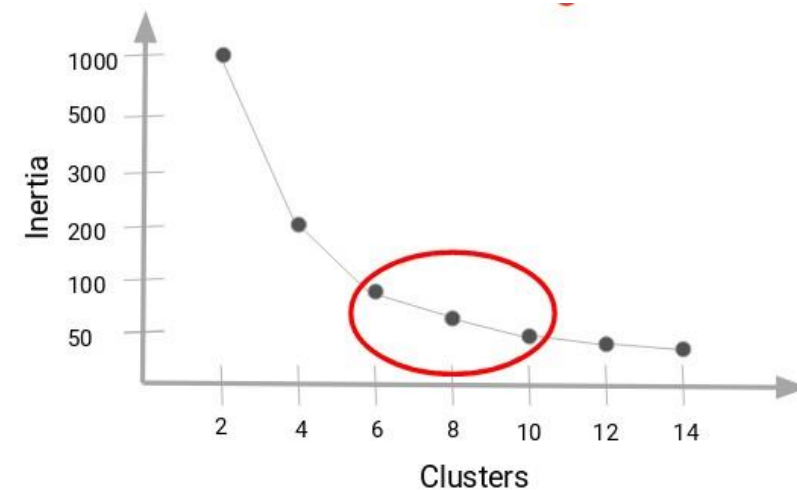
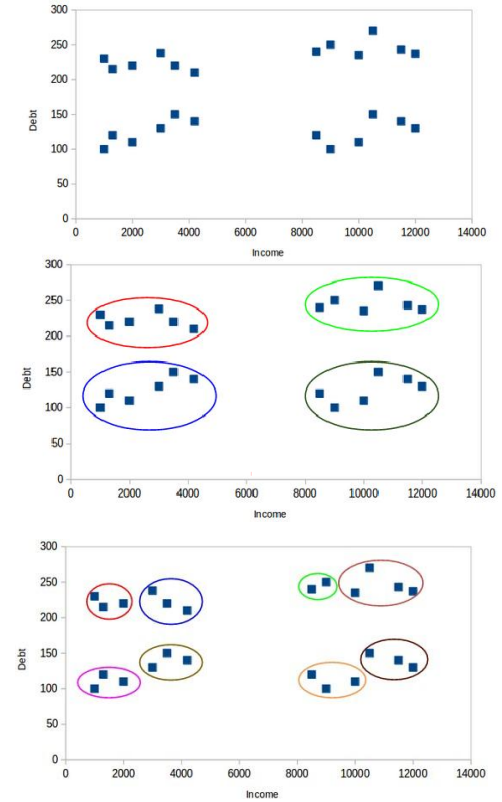
$$J = \sum_{j=1}^K \sum_{x_i \in C_j} \|x_i - \mu_j\|^2$$

- Where μ_j is the mean over all points in cluster C_j .
- Our goal is to minimize the aggregate intra-cluster distance
 - the same as reducing the total variance across each cluster if Euclidean distances are used
- Converges to a local minimum
 - Different starting points lead to different results
 - Iterate several times with different starting points
 - Pick the clustering that produces the smallest aggregate distance



Optimal Number of Clusters

- How many clusters?
 - Real world classes e.g. $K=10$ for digit recognition (0...9)
- How about optimizing the objective function for $K = 2, 3, \dots, n$? (Assuming there are n data points)
 - Run K-means, record J
 - Problem: J is minimized when $K = n$.
- Visually find the point:
 - where the mountain ends, rubble begins
 - Elbow method: Maximize second derivative of J
 - The point where the rate of decline changes the most



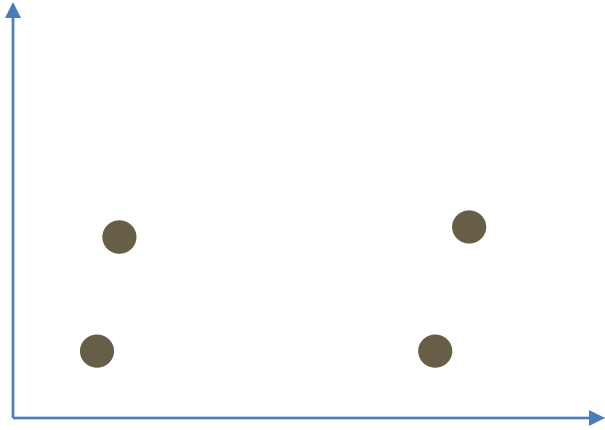
Evaluating Clustering

- Hard to figure out if a particular clustering good
- **Extrinsic evaluation**
 - Helps us solve another problem e.g. classification
 - Represent instances with cluster features and predict
 - Train a different classifier for each sub-population
 - Identify and eliminate outliers
- **Intrinsic evaluation**
 - Useful in and of itself
 - Qualitative: Understand the makeup of the data
 - Quantitative: Compare against tagged classes
 - Align and evaluate as classifier are evaluated
 - Compare with Human Judgement
 - Get a part of the dataset manually tagged and compare results (agreements)



Caveats

- Nearby points may not end up in the same cluster
 - Local minimum



- Does not preserve the property that nearby points end up in the same cluster
 - Agglomerative clustering preserves that



Categorical Data: K-modes

(K-Modes intuition and example, https://www.youtube.com/watch?v=b39_vipRkUo)

Algorithm

1. Pick K instances at random and use as clusters
2. Compare each instance with each cluster
 - +1 for each mismatch, 0 for each match
3. Assign each instance to the closest cluster
4. Each feature should have the mode (most common response) for each centroid
5. Repeat until convergence

Cluster#	Instances	Feature1	Feature2	Feature3	Feature4	Feature5	Dist(xi, c1)	Dist(xi, c2)	Dist(xi, c3)	Assigned
c1	x1	C	B	D	C	B	3	4	1	c3
	x2	B	B	A	C	D	4	4	3	c3
	x3	D	B	B	D	B	0	4	4	c1
	x4	B	E	C	A	C	5	4	5	c2
c2	x5	D	D	D	D	B	2	3	4	c1
	x6	C	D	A	D	C	4	0	4	c2
	x7	D	E	A	C	C	4	3	4	c2
c3	x8	D	C	B	B	B	2	5	5	c1
	x9	C	B	D	C	A	4	4	0	c3
	x10	B	A	B	D	A	3	4	4	c1



Categorical Data: K-modes

(K-Modes intuition and example, https://www.youtube.com/watch?v=b39_vipRkUo)

Algorithm

1. Pick K instances at random and use as clusters
2. Compare each instance with each cluster
 - +1 for each mismatch, 0 for each match
3. Assign each instance to the closest cluster
4. Each feature should have the mode (most common response) for each centroid
5. Repeat until convergence

Cluster#	Instances	Feature1	Feature2	Feature3	Feature4	Feature5	Dist(xi, c1)	Dist(xi, c2)	Dist(xi, c3)	Assigned
c1	x3	D	B	B	D	B	0	4	4	c1
	x5	D	D	D	D	B	2	3	4	c1
	x8	D	C	B	B	B	2	5	5	c1
	x10	B	A	B	D	A	3	4	4	c1
	x4	B	E	C	A	C	5	4	5	c2
c2	x6	C	D	A	D	C	4	0	4	c2
	x7	D	E	A	C	C	4	3	4	c2
	x1	C	B	D	C	B	3	4	1	c3
	x2	B	B	A	C	D	4	4	3	c3
c3	x9	C	B	D	C	A	4	4	0	c3
New c1		D	?	B	D	B				
New c2		?	E	A	?	C				
New c3		C	B	D	C	?				



Thank you!

